The PYPL ranking, which reports only the top 30 programming languages, is based on how frequently programming language tutorials are searched on Google. It therefore reflects learning demand rather than direct industry usage. This approach naturally favours beginner-friendly languages and those widely taught in academic institutions. The relatively high position of C illustrates this tendency.

The IEEE Spectrum ranking is the most comprehensive among the schemes considered, incorporating a wide range of indicators such as Google search trends, GitHub activity, Stack Overflow data, job postings, academic publications, and social media signals. By combining these diverse metrics, it attempts to balance perspectives from academia, industry, and the developer community. As a result, the IEEE Spectrum ranking is less biased towards any single measure and serves as a strong indicator of overall ecosystem relevance. Among the four ranking schemes, it is also the most closely aligned with industry demand. The relatively high ranking of Kotlin illustrates this tendency.

Finally, we consider the TIOBE index. Although it lists the largest number of programming languages, its rankings can sometimes be confusing and potentially misleading. The TIOBE index is based on measures of visibility, including search engine hits, indexed web pages, books, tutorials, courses, vendor materials, and job mentions. Its primary limitation is that it reflects online presence rather than actual industry usage. Therefore, older programming languages often appear much higher in the rankings than languages currently favoured in modern software development. For example, Perl is ranked 11th, Fortran 12th, and Ada 18th. I have seen a lot of things in my life, but I have yet to see a professional Ada programmer in person. Programming languages like Fortran and Ada have significant historical importance and remain relevant in specialised domains, but their high rankings do not necessarily reflect mainstream industry adoption.

We now turn to the trends emerging from our ranking. The most noticeable shift is the relatively weaker performance of PHP. In all previous editions, PHP consistently appeared in the top 10 across all four rankings considered. This time, however, it is placed in the top 10 in only two of them. At present, it is too early to draw firm conclusions about PHP's future. This may represent a short-term fluctuation, or it could signal a gradual decline in the language's popularity.

Another surprising finding is the absence of TypeScript from the Top 10 list. This outcome is largely driven by its low ranking—32nd—in the TIOBE index. In contrast, the IEEE Spectrum ranking places TypeScript in 7th position, just below JavaScript, highlighting its strong relevance in modern software development. The comparatively weak showing in the TIOBE index may be partly explained by TypeScript's relative youth (its first version was released in 2012) and by the likelihood that many TypeScript-related searches are counted under JavaScript.

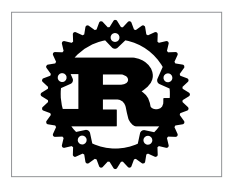

Figure 1: The logo of Rust

Rust's first entry into the Top 10 list is another important trend. Given that it was released only in 2012, this represents a significant achievement for the language. Figure 1 shows the logo of Rust.

Winners, losers, and the under-17 contenders in the programming language race

Now let us identify the languages that are gaining momentum. As in our previous rankings, the clear winner is Python. Its continued dominance is not surprising, even amid the rapid expansion of AI and quantum computing. In fact, these very trends reinforce Python's position. Python remains the primary language for AI development, and leading quantum computing toolkits such as Qiskit are built around it. Rather than slowing Python's progress, the explosive growth of AI applications and quantum simulations has accelerated its momentum and strengthened its leadership.

Another important trend is that, unlike 2019 or 2021, there are no clear losers. Back in 2019 I thought that Ruby and Perl might be doomed. But in just two years I had to retract my words about Ruby. My 2021 list had Perl and Objective-C as languages destined for oblivion. However, after five years I am retracting my prediction about Perl. Not only is Perl still around, it is also surviving even after Perl 6 was officially renamed Raku and is considered a programming language independent of Perl (Perl 5). Similarly, Objective-C has 1 Top 10 finish, 3 Top 20 finishes, and ranks 29th in the TIOBE index — not the signs of a language in decline. So, I have an empty list of languages that are losing the battle for dominance in the programming world.

Finally, we turn to the under-17 contenders in the race for the programming language championship. The original 2019 edition focused on languages first released in 2009 or later. Today, these languages have matured, yet even the oldest among them is no more than 17 years old. The list compiled in 2019 included Swift, TypeScript, Go, Rust, Kotlin, Dart, and Julia. With the addition of Solidity in 2021, it expanded to eight promising programming languages.

What trends can we infer for 2026 from the performance of these languages? Swift, TypeScript, Go, Rust, Kotlin, and Dart each record at least three Top 20 finishes, indicating strong and sustained visibility across rankings. This consistency suggests that they have secured firm positions